

Reviewer Pack — ε -Approximation Complexity Lower Bound for k-CLIQUE via Mono...

Entry 5c00c6af4f62 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 02:10:10 UTC

ε -Approximation Complexity Lower Bound for k-CLIQUE via Monotone DNF Size

Entry ID: 5c00c6af4f62 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 02:10:10 UTC

1. Conjecture

Field A (mathematical branch): Approximation Theory of Boolean Functions **Field B** (complexity object): Monotone DNF Formulas for k-CLIQUE

Statement:

For any $\varepsilon \in (0, 1/2)$, the ε -approximation complexity of the k-CLIQUE function is $\Omega(n)$ when $k \geq \log n$, and this complexity is submodular under conjunction of functions.

Rationale (proposer's reasoning):

Approximation theory provides tools to analyze how well boolean functions can be approximated by simpler functions, which could reveal structural limitations in monotone DNF representations. The submodularity under conjunction suggests that combining functions increases the required complexity, aligning with known lower bounds for k-CLIQUE.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cc6258a455c6f887

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_clique(n, k):
        clique = set()
        while len(clique) < k:
            node = random.randint(0, n-1)
            if all(node not in edge for edge in clique):
                clique.add(node)
        return clique

    def dnf_approximation_size(clique, epsilon):
        n = len(clique)
        size = 2**n
        while True:
            approx_clique = set()
            for _ in range(n):
                node = random.choice(list(clique))
                if all(node not in edge for edge in approx_clique):
                    approx_clique.add(node)
            if len(approx_clique) >= (1 - epsilon) * n:
                return size
            size *= 2

    n_values = [5, 10, 15, 20, 30, 40]
    total_size = 0
    instances_tested = 0

    for n in n_values:
        for _ in range(5): # Ensure at least 30 instances per seed
            k = max(int(math.log(n)), 1)
            clique = generate_k_clique(n, k)
            size = dnf_approximation_size(clique, 0.1)
            total_size += size
            instances_tested += 1

    mean_size = total_size / instances_tested
```

```

conjecture_holds = mean_size >= n_values[-1]

return {
    "metric_name": "DNF Size",
    "metric_value": mean_size,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else "mapping_undefined"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e72ffab8.py", line 70, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e72ffab8.py", line 49, in run_trial
    clique = generate_k_clique(n, k)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e72ffab8.py", line 25, in generate_k_clique
    if all(node not in edge for edge in clique):
        ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e72ffab8.py", line 25, in <genexpr>
    if all(node not in edge for edge in clique):
        ~~~~~
TypeError: argument of type 'int' is not iterable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with TypeError due to non-iterable integer in membership check, preventing data collection | next: Fix the generate_k_clique function to ensure edges are iterable structures (e.g., tuples) before membership testing

11. Audit log (LLM calls)

Total LLM calls: 14

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	61692
2	propose	ollama_remote	qwen3:8b	0	0	139111
3	propose	ollama_remote	qwen3:8b	0	0	149904
4	propose	ollama_remote	qwen3:8b	0	0	137894
5	propose	ollama_remote	qwen3:8b	0	0	79111
6	propose	ollama_remote	qwen3:8b	0	0	85603
7	preregistration	ollama_remote	qwen3:8b	0	0	31886
8	novelty	ollama_remote	qwen3:8b	0	0	27254
9	novelty	ollama_remote	qwen3:8b	0	0	19986
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11475
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6981
12	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8451
13	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8821
14	judge	ollama_remote	qwen3:8b	0	0	27628

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 795799 ms total latency. Provider mix: {'ollama_remote': 14}

(full prompt+response transcripts available in `research/audit/5c00c6af4f62.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5c00c6af4f62.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5c00c6af4f62.tar.gz` (if generated)